RICA Multi-Agent — Before & After

Date: 2026-04-16 **Reference:** RICA Multi-Agent Technical Document v4.0 for full implementation details

---

What is RICA?

RICA is an internal AI coding assistant used by 1,000+ engineers. It runs as a VS Code extension, connecting to our internal LLM APIs (Rica/Databricks) to help developers read, write, debug, and review code — all without leaving the IDE.

RICA is built as a fork of Continue, the open-source AI coding assistant. We extended it significantly with multi-agent orchestration, token tracking, and production-grade agent infrastructure.

Before: Chat & Agent Mode

When RICA launched, it had two modes. Both were useful but fundamentally limited — a single LLM doing everything by itself, one step at a time.

Chat Mode

The simplest interaction. The user asks a question, the LLM responds with text. No access to the codebase — the model can only work with what the user pastes into the conversation.

- No tool access — cannot read files, search code, or run commands
- No context beyond what the user provides
- Good for: explaining concepts, answering quick questions, brainstorming approaches
- Limitation: the model often says "I'd need to see the code" but can't actually look at it

Agent Mode

A step up from chat. The LLM gains access to 12 built-in tools — it can read files, edit code, search the codebase, and run terminal commands. It works like a junior developer: read some code, think about it, make a change, check if it works.

- **Sequential execution** — one tool call at a time, strictly ordered. The LLM reads a file, waits for the result, decides what to do next, makes an edit, waits again, runs a test, waits again.
- **Single context window** — the entire conversation (system prompt + user message + all tool results + all LLM responses) lives in one context window. For complex tasks touching many files, this fills up fast.
- **No specialization** — the same LLM with the same system prompt handles investigation, implementation, and testing. There's no distinction between "figure out where the bug is" and "write the fix."

- Good for: focused, single-file tasks — fix a specific bug, add a function, update a config

What Was Missing

Area	Problem	Impact
Parallelism	All tool calls execute one after another	A 10-file review takes 10x as long as it should — the LLM reads file 1, analyzes it, then reads file 2, analyzes it, etc.
Task decomposition	The user must manually break down complex tasks	"Review and fix all security issues" becomes a multi-hour back-and-forth where the user guides each step
Specialization	Same prompt for reading, writing, testing, reviewing	The model doesn't optimize its approach per task type — a code reviewer doesn't need edit permissions, a tester shouldn't be writing production code
Context scalability	One context window for everything	Complex tasks that touch 15+ files exhaust the context window. The model forgets earlier findings as new tool results push older messages out
Visibility	Users can't see what the agent is doing or how much it costs	The agent silently consumes tokens with no breakdown. Users don't know if it's stuck, wasting time, or making progress
Error recovery	If a tool call fails, the model must handle it ad-hoc	No structured retry mechanism. A rate limit or timeout kills the entire task

---

After: Multi-Agent Mode

RICA now supports a third mode — **Multi-Agent** — where an orchestrator LLM decomposes tasks and delegates to specialized sub-agents.

How It Works

```
User: "Review all files in src/ for security issues and fix them"

Orchestrator LLM
  ├── Analyzes task (builtin_analyze_task)
  │   └── "3 files detected, no overlaps, parallel is safe"
  │
  ├── Spawns agents in parallel (builtin_spawn_agents)
  │   ├── [Reviewer]  Security audit of src/auth.ts
  │   ├── [Reviewer]  Security audit of src/api.ts
  │   └── [Reviewer]  Security audit of src/db.ts
  │
  ├── Receives all results
  │   └── "Found SQL injection in db.ts line 42, XSS in api.ts line 78"
```

```
   │
   ├── Spawns fix agent
   │   └── [Implementer]  Fix security issues in db.ts and api.ts
   │
   └── Summarizes: "Fixed 2 security vulnerabilities. Here's what changed..."
```

5 Execution Patterns

The orchestrator automatically selects the best pattern based on the task:

Pattern	When	Example
Single Agent	Small, focused task (≤5 files)	"Fix the login bug"
Parallel	Independent subtasks	"Review auth, update README, fix CSS"
Sequential	Dependent phases	"Find the bug, then fix it"
Iterative	Test-fix-retest loops	"Fix failing tests" (implement → test → retry)
Collaborative	Multi-perspective review	"Review payment module for production readiness"

6 Specialist Roles

Each sub-agent gets a role-specific system prompt and tool access:

Role	What it does	Can edit files?
Researcher	Explores codebase, searches for patterns, reads files	No
Implementer	Writes code, edits files, makes changes	Yes
Tester	Runs tests, validates changes	No
Reviewer	Code review, security audit, quality assessment	No
Analyst	Investigates bugs, traces root causes	No
Planner	Creates implementation plans, architecture decisions	No

Safety: Conflict Detection

When multiple agents work in parallel, RICA prevents file conflicts at two levels:

Pre-flight — Before spawning agents, the task analyzer detects if two subtasks target the same file and warns the orchestrator to merge them or run sequentially.

Post-flight — After agents complete, the system checks if multiple agents edited the same file and warns the orchestrator with the edit timeline.

Real-Time Visibility

- **Animated robot icons** show agents working in parallel
- **Floating popup card** with per-agent status, role badges, and expandable task details
- **Token tracking** per agent and per session — prompt, completion, reasoning, cache tokens
- **Per-agent cancellation** — cancel individual agents or kill all at once

---

Side-by-Side Comparison

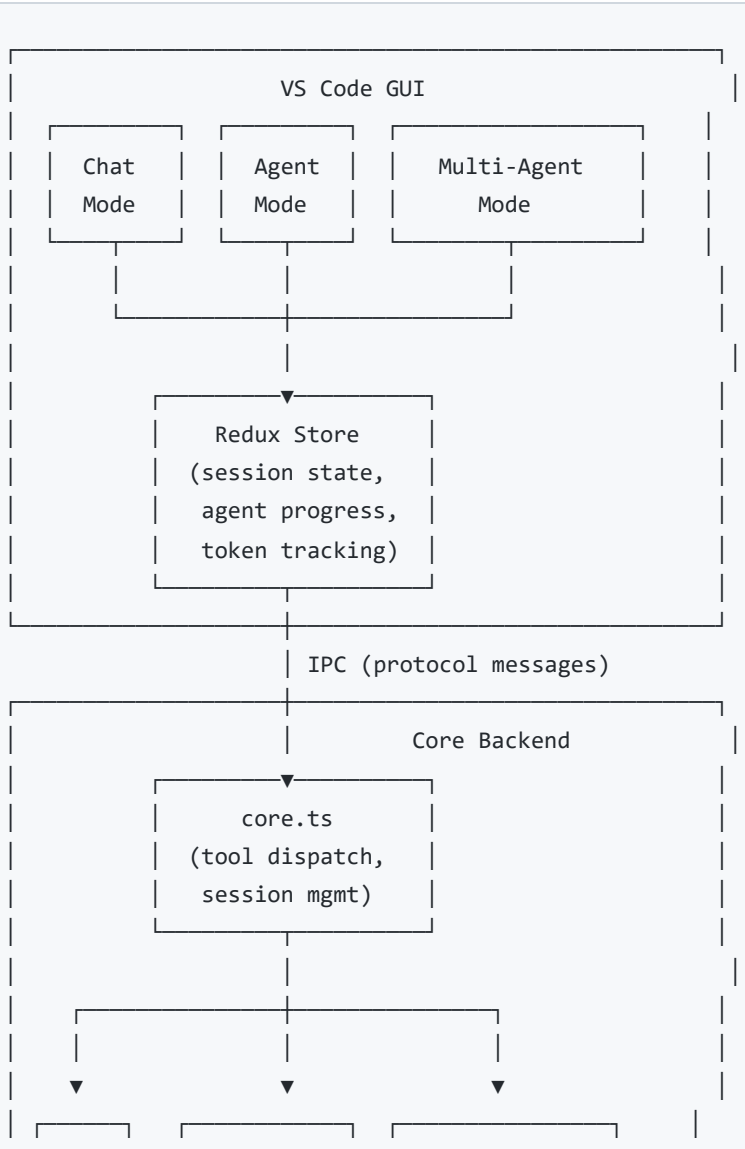
Capability	Chat	Agent	Multi-Agent
Text responses	Yes	Yes	Yes
Tool access	No	Yes	Yes (per agent)
Parallelism	No	No	Yes (Promise.all)
Specialization	No	No	6 roles
Task decomposition	No	No	Automatic
Conflict detection	N/A	N/A	Pre-flight + Post-flight
Retry on failure	No	No	Automatic (3 retries)
Token visibility	No	Basic	Full (per-agent + session)
Cancel granularity	Stop stream	Stop stream	Per-agent, per-session, or all
Context sharing	N/A	N/A	ContextBridge between phases
Task analysis	No	No	builtin_analyze_task
Max agents	1	1	10 per batch

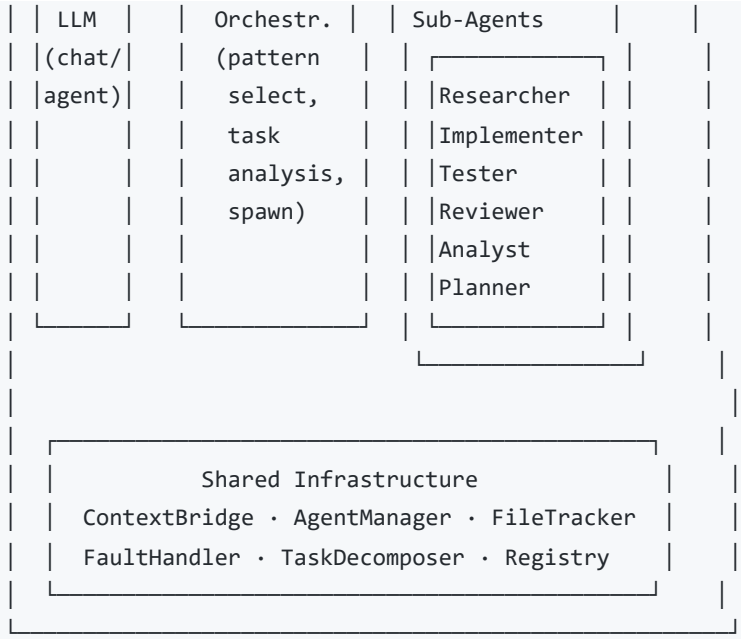
When to Use Each Mode

Use Case	Recommended Mode
"Explain this function"	Chat

Use Case	Recommended Mode
"What does this error mean?"	Chat
"Fix the null pointer in auth.ts"	Agent
"Add a login form to the settings page"	Agent
"Review all files in src/ for bugs"	Multi-Agent
"Refactor the auth system and update tests"	Multi-Agent
"Audit security, performance, and code quality"	Multi-Agent (Collaborative)
"Find the failing test and fix it"	Multi-Agent (Iterative)

Architecture at a Glance





Key Numbers

Metric	Value
New files added (v1.0 → v4.0)	22
Files modified	40
Total new code	~5,900 lines
Agent roles	6
Execution patterns	5
Max parallel agents	10
Max tool iterations per agent	25
Automatic retries on failure	3

For implementation details, data flows, type definitions, and technical specifications, see the RICA Multi-Agent Technical Document v4.0.